QUESTION 1

Problem 1 : Initialize and Configure Git

1. Create a new folder named **ExamRepo**.
2. Inside the folder, configure Git with your name and email.
3. Initialize the folder as a Git repository.
4. Show the current repository status.

Problem 2 : Tracking Files and Creating Commits

1. Create a new file named **notes.txt** and add the text:
   “Git is a version control system.”
2. Check the status of your repository.
3. Stage the file.
4. Commit the file with the message:
   “Added notes.txt”
5. Show the commit history.

Problem 3 : Modifying Files and Using Reset

1. Edit **notes.txt** by adding a second line:
   “Branches allow parallel development.”
2. Stage and commit this change with a suitable message.
3. View the commit history again.
4. Now **reset the repository to the first commit**.
5. Display the contents of **notes.txt** to show that the second line is removed.

Problem 4 : Creating and Switching Branches

1. Create a new branch named **feature-update**.
2. Switch to the **feature-update** branch.
3. Create a new file named **update.txt** containing:
   “This is a feature update.”
4. Stage and commit the file.
5. Show the commit history of the branch.

Problem 5 : Merging Branches

1. Switch back to the **master** branch.
2. Display the contents of the repository to show that **update.txt** does NOT exist.
3. Merge the **feature-update** branch into **master**.
4. Show that **update.txt** now exists in **master**.
5. Show the updated commit history.

Problem 6 : Creating Another Branch and Simulating Parallel Work

1. Create a branch named **design-change**.
2. Switch to the branch.
3. Modify **notes.txt** by adding:
   “This line was added on the design-change branch.”
4. Stage and commit the change.
5. Switch back to **master**.
6. Modify **notes.txt** again, adding:
   “This line was added on the main branch.”
7. Stage and commit this change.
8. Try merging **design-change** into **master** and observe what happens.

QUESTION 2

Problem 1 : Setup and First Commit

1. Create a folder named **ProjectAlpha**.
2. Initialize it as a Git repository.
3. Configure your Git username only (email not needed for this question).
4. Create a file named **readme.md** containing:
   "This is Project Alpha."
5. Stage and commit the file with the message:
   "Initial commit with readme.md"
6. See the commit history.

Problem 2 : Multiple Files and History Tracking

1. Create two files:
   - **app.txt**
   - **info.txt**
2. Add any content inside both files.
3. Stage **only app.txt**.
4. Check repository status and observe the difference.
5. Commit **app.txt** with a meaningful message.
6. Now stage and commit **info.txt** using a separate commit.
7. View commit history and explain how many commits exist and what they represent.

Problem 3 : Reverting to a Previous Version

1. Edit **readme.md** by adding:
   "Updated project description."
2. Stage and commit this change.
3. Now view the commit history and copy the commit ID of the **first commit**.
4. Revert back to the first commit.
5. Show that both **app.txt** and **info.txt** are now removed.
6. Show the contents of **readme.md** to confirm it has reverted.

Problem 4 : Creating Branches for New Features

1. Create a branch named **documentation**.
2. Switch to this branch.
3. Recreate **info.txt** and add the text:
   “Documentation in progress.”
4. Stage and commit the file.
5. View the log to verify the commit exists **only** in the documentation branch.

Problem 5 : Branch Modification & Merge

1. Switch back to the **master** branch.
2. Create a new branch named **feature-ui**.
3. Switch to **feature-ui**.
4. Create **ui.txt** containing:
   “Initial UI layout.”
5. Stage and commit the file.
6. Switch to the **master** branch.
7. Merge **documentation** into **master**.
8. Then merge **feature-ui** into **master**.
9. Verify that both **info.txt** and **ui.txt** now exist in master.

Problem 6 : Parallel Changes and Merge Behavior

1. On the **master** branch, edit **readme.md** to add:
   “Main branch update.”
2. Stage and commit this change.
3. Switch to **feature-ui** (which has already been merged earlier).
4. Edit **readme.md** here to add:
   “UI branch update.”
5. Stage and commit the change.
6. Switch back to **master**.
7. Attempt to merge **feature-ui** into **master** again.
8. Observe and explain what Git reports.
